



Super Admin Authentication Plan for Orviohub

1. Authentication approach

Since you're the only super admin initially, use a **simple but secure** approach:

Option A: Direct credentials (Recommended for now)

```
admin.orviohub.localhost/login
↓
Enter email and password
↓
Verify against platform admin table
↓
Create admin session
↓
Redirect to admin dashboard
```

Why this works:

- Simple to implement
- No overhead
- Easy to secure
- Can upgrade to SSO later

Option B: Use existing Orviohub account

```
admin.orviohub.localhost/login
↓
Sign in with your Orviohub account
↓
Check if user has platform admin role
↓
Grant admin access
↓
Redirect to admin dashboard
```

Why this works:

- Single account for everything
- Reuses existing auth system

- Easier to manage

Recommendation: Start with Option A (separate admin credentials) for security separation, then migrate to Option B later when the main auth system is stable.

2. Database schema

Add these tables to your backend:

platform_admins **table**

```
CREATE TABLE platform_admins (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  name VARCHAR(255) NOT NULL,  
  role VARCHAR(50) NOT NULL DEFAULT 'super_admin',  
  is_active BOOLEAN NOT NULL DEFAULT true,  
  last_login_at TIMESTAMP WITH TIME ZONE,  
  last_login_ip VARCHAR(45),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);  
  
CREATE INDEX idx_platform_admins_email ON platform_admins(email);
```

admin_sessions **table**

```
CREATE TABLE admin_sessions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  admin_id UUID NOT NULL REFERENCES platform_admins(id) ON DELETE CASCADE,  
  session_token VARCHAR(255) UNIQUE NOT NULL,  
  ip_address VARCHAR(45),  
  user_agent TEXT,  
  expires_at TIMESTAMP WITH TIME ZONE NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  last_active_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);  
  
CREATE INDEX idx_admin_sessions_token ON admin_sessions(session_token);  
CREATE INDEX idx_admin_sessions_admin ON admin_sessions(admin_id);  
CREATE INDEX idx_admin_sessions_expires ON admin_sessions(expires_at);
```

admin_audit_logs **table**

```
CREATE TABLE admin_audit_logs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  admin_id UUID REFERENCES platform_admins(id),  
  action VARCHAR(100) NOT NULL,  
  resource_type VARCHAR(50),
```

```
resource_id UUID,  
details JSONB,  
ip_address VARCHAR(45),  
user_agent TEXT,  
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);  
  
CREATE INDEX idx_admin_audit_admin ON admin_audit_logs(admin_id);  
CREATE INDEX idx_admin_audit_action ON admin_audit_logs(action);  
CREATE INDEX idx_admin_audit_created ON admin_audit_logs(created_at);
```

3. Authentication flow

Login

POST /api/v1/admin/auth/login

Request:

```
{  
  "email": "admin@orv.iohub.com",  
  "password": "your-secure-password"  
}
```

Response (success):

```
{  
  "admin": {  
    "id": "uuid",  
    "email": "admin@orv.iohub.com",  
    "name": "Your Name",  
    "role": "super_admin"  
  },  
  "session": {  
    "token": "secure-token",  
    "expiresAt": "2026-08-29T10:31:00Z"  
  }  
}
```

Response (error):

```
{  
  "error": "Invalid credentials"  
}
```

Session validation

GET /api/v1/admin/auth/me

Headers:

Authorization: Bearer <session-token>

Response (valid):

```
{  
  "admin": {
```

```
    "id": "uuid",
    "email": "admin@orviohub.com",
    "name": "Your Name",
    "role": "super_admin"
  }
}

Response (invalid):
{
  "error": "Unauthorized"
}
```

Logout

```
POST /api/v1/admin/auth/logout
Headers:
  Authorization: Bearer <session-token>

Response:
{
  "success": true
}
```

Refresh session

```
POST /api/v1/admin/auth/refresh
Headers:
  Authorization: Bearer <session-token>

Response:
{
  "session": {
    "token": "new-secure-token",
    "expiresAt": "2026-08-29T10:31:00Z"
  }
}
```

4. Security requirements

Password requirements

- Minimum 12 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one number
- At least one special character
- Password history (prevent reuse of last 5 passwords)

Session management

- Session timeout: 8 hours of inactivity
- Absolute session expiry: 24 hours
- Secure cookies (HttpOnly, SameSite=Strict, Secure)
- Session rotation on each request
- Automatic logout on password change
- Single active session (optional, for maximum security)

Rate limiting

- Login attempts: 5 per minute per IP
- Failed logins: Lock account after 10 failed attempts
- Cooldown period: 15 minutes after logout

Additional security

- Require HTTPS in production
- Log all admin actions
- Log all login attempts
- Alert on suspicious activity
- IP whitelist (optional, for your specific IP)
- Two-factor authentication (add later)

5. Frontend setup (admin dashboard)

Protected routes

```
// apps/admin/src/App.tsx

<Routes>
  <Route path="/login" element={<Login />} />
  <Route path="/forgot-password" element={<ForgotPassword />} />

  <Route element={<AdminLayout />>
    <Route path="/dashboard" element={<Dashboard />} />
    <Route path="/users" element={<Users />} />
    <Route path="/organizations" element={<Organizations />} />
    <Route path="/billing" element={<Billing />} />
    <Route path="/settings" element={<Settings />} />
  </Route>
</Routes>
```

Auth guard

```
// apps/admin/src/components/AuthGuard.tsx

function AuthGuard({ children }: { children: React.ReactNode }) {
  const { isAuthenticated, isLoading } = useAuth();

  if (isLoading) {
    return <LoadingSpinner />;
  }

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  return <>{children}</>;
}
```

Auth context

```
// apps/admin/src/contexts/AuthContext.tsx

interface Admin {
  id: string;
  email: string;
  name: string;
  role: 'super_admin';
}

interface AuthContextType {
  admin: Admin | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  login: (email: string, password: string) => Promise<void>;
  logout: () => Promise<void>;
}
```

6. Backend implementation

Admin auth middleware

```
// backend/src/middleware/adminAuth.ts

export async function adminAuthMiddleware(
  fastify: FastifyInstance,
  request: FastifyRequest,
  reply: FastifyReply
) {
  const token = request.headers.authorization?.replace('Bearer ', '');
}
```

```

if (!token) {
  throw new UnauthorizedError('No token provided');
}

const session = await fastify.db.query(
  `SELECT * FROM admin_sessions
   WHERE session_token = $1
   AND expires_at > NOW()`,
  [token]
);

if (session.rows.length === 0) {
  throw new UnauthorizedError('Invalid or expired session');
}

const admin = await fastify.db.query(
  `SELECT id, email, name, role FROM platform_admins
   WHERE id = $1 AND is_active = true`,
  [session.rows[0].admin_id]
);

if (admin.rows.length === 0) {
  throw new UnauthorizedError('Admin not found or inactive');
}

request.admin = {
  id: admin.rows[0].id,
  email: admin.rows[0].email,
  name: admin.rows[0].name,
  role: admin.rows[0].role,
};

// Update last active
await fastify.db.query(
  `UPDATE admin_sessions
   SET last_active_at = NOW()
   WHERE session_token = $1`,
  [token]
);
}

```

Login endpoint

```

// backend/src/routes/admin/auth.ts

fastify.post('/login', async (request, reply) => {
  const { email, password } = request.body;

  // Find admin
  const admin = await fastify.db.query(
    `SELECT * FROM platform_admins WHERE email = $1`,
    [email]
  );

  if (admin.rows.length === 0) {

```

```

    throw new UnauthorizedError('Invalid credentials');
  }

  // Verify password
  const valid = await bcrypt.compare(password, admin.rows[0].password_hash);

  if (!valid) {
    throw new UnauthorizedError('Invalid credentials');
  }

  // Check if active
  if (!admin.rows[0].is_active) {
    throw new UnauthorizedError('Account is deactivated');
  }

  // Create session
  const sessionToken = crypto.randomBytes(32).toString('hex');
  const expiresAt = new Date(Date.now() + 24 * 60 * 60 * 1000); // 24 hours

  await fastify.db.query(
    `INSERT INTO admin_sessions (admin_id, session_token, expires_at, ip_address, user_agent)
    VALUES ($1, $2, $3, $4, $5)`,
    [
      admin.rows[0].id,
      sessionToken,
      expiresAt,
      request.ip,
      request.headers['user-agent'],
    ]
  );

  // Log login
  await fastify.db.query(
    `INSERT INTO admin_audit_logs (admin_id, action, ip_address, user_agent)
    VALUES ($1, 'login', $2, $3)`,
    [admin.rows[0].id, request.ip, request.headers['user-agent']]
  );

  // Update last login
  await fastify.db.query(
    `UPDATE platform_admins
    SET last_login_at = NOW(), last_login_ip = $1
    WHERE id = $2`,
    [request.ip, admin.rows[0].id]
  );

  return {
    admin: {
      id: admin.rows[0].id,
      email: admin.rows[0].email,
      name: admin.rows[0].name,
      role: admin.rows[0].role,
    },
    session: {
      token: sessionToken,
      expiresAt: expiresAt,
    }
  };
}

```



```
    },  
  };  
});
```

7. Initial admin setup

Create your admin account

```
// backend/src/scripts/create-admin.ts  
  
import bcrypt from 'bcrypt';  
  
async function createAdmin() {  
  const email = 'admin@orv.iohub.com';  
  const password = 'your-secure-password';  
  const passwordHash = await bcrypt.hash(password, 12);  
  
  await db.query(  
    `INSERT INTO platform_admins (email, password_hash, name, role)  
    VALUES ($1, $2, $3, $4)`,  
    [email, passwordHash, 'Your Name', 'super_admin']  
  );  
  
  console.log('Admin account created');  
}  
  
createAdmin();
```

Run this once during initial setup.

8. Admin dashboard routes

Protected routes

All admin routes should require authentication:

```
// backend/src/routes/admin/index.ts  
  
fastify.register(adminAuthMiddleware, { prefix: '/admin' });  
  
// Dashboard  
fastify.get('/admin/dashboard', getDashboard);  
  
// Users  
fastify.get('/admin/users', getUsers);  
fastify.get('/admin/users/:id', getUser);  
fastify.patch('/admin/users/:id/suspend', suspendUser);  
  
// Organizations
```

```

fastify.get('/admin/organizations', getOrganizations);
fastify.get('/admin/organizations/:id', getOrganization);
fastify.patch('/admin/organizations/:id/suspend', suspendOrganization);

// Billing
fastify.get('/admin/billing/subscriptions', getSubscriptions);
fastify.post('/admin/billing/adjust', adjustBilling);

// Audit logs
fastify.get('/admin/audit-logs', getAuditLogs);

```

9. Audit logging

Log every admin action:

```

// Example: Suspend user

async function suspendUser(adminId: string, userId: string) {
  await db.query(
    `UPDATE users SET status = 'suspended' WHERE id = $1`,
    [userId]
  );

  await db.query(
    `INSERT INTO admin_audit_logs (admin_id, action, resource_type, resource_id, details)
    VALUES ($1, 'user_suspended', 'user', $2, $3, $4)`,
    [adminId, userId, JSON.stringify({ userId }), request.ip]
  );
}

```

10. Environment variables

```

# .env

# Admin auth
ADMIN_JWT_SECRET=your-secret-key
ADMIN_SESSION_EXPIRY=86400000
ADMIN_PASSWORD_MIN_LENGTH=12

# Database
DATABASE_URL=postgresql://...

# Security
ADMIN_RATE_LIMIT_WINDOW=60000
ADMIN_RATE_LIMIT_MAX=5

```

11. Future enhancements

Add these later:

1. Two-factor authentication (2FA)

- TOTP (Google Authenticator)
- SMS verification
- Email verification

2. IP whitelist

- Only allow login from specific IPs

3. Session management UI

- View active sessions
- Revoke sessions

4. Password rotation

- Force password change every 90 days

5. Multiple admin roles

- Super admin
- Support admin
- Billing admin

6. SSO integration

- Use your main Orvionhub account
- OAuth with Google/Microsoft

12. Summary

For now, implement:

1. ✓ Separate admin project (apps/admin)
2. ✓ `platform_admins` table
3. ✓ `admin_sessions` table
4. ✓ `admin_audit_logs` table
5. ✓ Login endpoint (POST `/api/v1/admin/auth/login`)
6. ✓ Session validation endpoint (GET `/api/v1/admin/auth/me`)
7. ✓ Logout endpoint (POST `/api/v1/admin/auth/logout`)
8. ✓ Admin auth middleware
9. ✓ Protected admin routes
10. ✓ Frontend login page
11. ✓ Auth guard for protected routes

12. ✓ Initial admin account creation script

This gives you a secure, simple admin authentication system that you can expand later.